



ANSWERED



CORRECT



INCORRECT



FLAGGED

0 / 70

0%

SUBMIT ALL & SCORE

REVEAL ALL ANSWERS

RESET EXAM

PRINT / SAVE PDF

NEXT UNANSWERED ↓

ALL (70)

COMPOSITION (16)

CONTENT AGGREGATION (7)

CUSTOMIZING USD (4)

DATA EXCHANGE (11)

DATA MODELING (9)

DEBUGGING (8)

PIPELINE DEV (10)

VISUALIZATION (5)

COMPOSITION

#1

COMPOSITION · 23% EXAM WEIGHT

In USD, the acronym LIVERPS describes the strength ordering of composition arcs. Which of the following correctly represents the order from strongest to weakest?

A Local opinions → Inherits → Variants → References → Payloads → Specializes

B Local opinions → Inherits → Variants → References → Specializes → Payloads

C Variants → Local opinions → Inherits → References → Payloads → Specializes

D Inherits → Local opinions → Variants → Specializes → References → Payloads

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: LIVERPS stands for: **L**ocal opinions, **I**nherits, **V**ariants, **E**externally-referenced (References), **P**ayloads, **S**pecializes — from strongest to weakest. Payloads come after



References and before Specializes.

#2

COMPOSITION · 23% EXAM WEIGHT

A shot layer references a character asset using `prepend references`. An animator then directly authors a translate value on the referenced prim inside the shot layer. According to LIVERPS, which opinion wins?

- ☐ A The translate value inside the reference, because it was authored first
- ☒ B The translate value in the shot layer, because local opinions are stronger than references
- ☐ C Neither—USD does not allow local overrides on referenced prims
- ☐ D The reference wins because `prepend` gives it higher priority than local opinions

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Local opinions (L) are the strongest arc in LIVERPS, sitting above inherits, variants, references, payloads, and specializes. A value authored directly in the shot layer on a referenced prim will always override any opinion from the reference itself.

#3

COMPOSITION · 23% EXAM WEIGHT

A pipeline team wants multiple artists to work simultaneously on the same character asset—one on geometry, one on materials, and one on rigging—without overwriting each other. Which composition mechanism is best suited for this?

- ☐ A Variant sets, with each artist in a different variant
- ☒ B Sublayers within the asset, with each workflow in a separate layer
- ☐ C Separate references from a shared parent stage file
- ☐ D Payloads, with each workflow deferred until needed

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Sublayers allow multiple layers to be stacked within the same stage or asset. Each artist can work in their own sublayer (geometry.usda, materials.usda, rig.usda) and opinions are composed top-to-bottom. This is the standard pattern for modeling multi-user collaborative workstreams within a single asset.

#4

COMPOSITION · 23% EXAM WEIGHT

When referencing a USD layer that contains multiple root prims, USD needs to know which prim to use as the target. What is the correct way to specify this?

- ☐ A Set the ``rootPrim`` metadata on the referencing prim
- ☐ B Add a ``primPath`` argument to the reference arc
- ☒ C Set the ``defaultPrim`` metadata in the referenced layer's file header
- ☐ D Rename all prims to match the referencing prim's name

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: defaultPrim metadata, set in the layer's header (e.g., ``defaultPrim = "Robot"``), tells USD which root prim to use when the layer is referenced without an explicit prim path. If omitted, the reference must supply an explicit prim path—otherwise composition will fail.

#5

COMPOSITION · 23% EXAM WEIGHT

You author a variant set ``surfacing`` on a prop asset with options ``wet`` and ``dry``. An FX artist wants to override just the wet variant's emissive value in a downstream shot. Which specifier should they use when targeting the relevant prim inside the variant?

- ☐ A ``def`` — to redefine the prim completely in the shot layer
- ☐ B ``class`` — because class prims propagate across the whole stage
- ☒ C ``over`` — to add non-destructive opinions on top of the existing definition

C

D `payload` — to load the override only when needed

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``over`` is a specifier that authors opinions on a prim without requiring it to exist in the current layer. It is the correct way to non-destructively add or change attributes on existing prims (including those inside variants) from a downstream or shot layer.

#6

COMPOSITION · 23% EXAM WEIGHT

A technical director sets ``prepend references`` on a prim that already has ``append references``. What is the practical effect of using ``prepend`` instead of ``append``?

A Prepended references are loaded before appended ones and have higher opinion strength

B Prepended references are ignored if any appended reference exists

C Prepend makes the reference load on demand, while append loads it eagerly

D Append and prepend are functionally identical for references

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: List editing keywords like `prepend` and `append` control the order of items in lists. A prepended reference appears earlier in the reference list, meaning it has stronger opinions than an appended reference on the same prim. This matters when multiple references contribute opinions to the same attribute.

#7

COMPOSITION · 23% EXAM WEIGHT

An asset uses a payload arc to load its geometry. A downstream pipeline stage wants to process metadata about the asset (bounding box, kind, purpose) without loading the full geometry. What is the correct behavior?

A

Payloads must always be loaded before any metadata is accessible

B

Metadata and properties outside the payload are accessible even when the payload is unloaded

C

The prim disappears entirely from the stage when its payload is unloaded

D

Payloads can never be unloaded once a stage is opened

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Payloads are specifically designed for deferred loading of heavy data. Prim identity, metadata (kind, purpose, extentsHint), and properties defined *outside* the payload arc remain accessible on an unloaded prim. This is the core advantage over references—payloads let pipelines manage working sets efficiently.

#8

COMPOSITION · 23% EXAM WEIGHT



A prop has a variant set ``lod`` with values ``high``, ``mid``, and ``low``. A set-dressing stage selects ``mid``, but a shot layer wants to override this to ``low`` for performance. Where should the variant selection for the shot be authored?

A

Inside the prop's own USD file, replacing the default selection

B

In the set-dressing layer using a ``prepend sublayers`` to the shot

C

Directly on the referencing prim in the shot layer using the ``variants`` metadata

D

In a separate payload layer that references the prop

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Variant selections are authored as **metadata on the referencing prim** (``variants = { string lod = "low" }``). A stronger layer (e.g., the shot layer) can override a weaker layer's variant selection because variant selection opinions follow LIVERPS strength ordering.

#9

COMPOSITION • 23% EXAM WEIGHT



Two USD layers—`base.usda` and `override.usda`—both define ``color3f[]`` `primvars:displayColor`` on ``/World/Cube``. The stage's root layer sublayers `override.usda` on top of `base.usda`. Which value appears on the composed stage?

A

The value from `base.usda`, because it was authored first

B

Both values are merged into an array

C

The value from `override.usda`, because it is in a stronger (higher) sublayer

D

USD raises a composition error and uses a fallback default

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In a sublayer stack, layers listed **earlier (higher)** have stronger opinions. The root layer is strongest, and sublayers are ordered by their position in the ``subLayers`` list—earlier entries override later ones. ``override.usda`` being on top means its value wins for ``primvars:displayColor``.

#10

COMPOSITION • 23% EXAM WEIGHT



An asset producer wants to share a character with an external vendor, but the character's USD file contains internal pipeline paths in its references. Which operation is most appropriate to prepare it for external delivery?

A

Export the stage using ``UsdStage::Export()`` to write a single USDC file

B

Flatten the stage to a single layer and rewrite asset paths using a USDZ package

C

Use sublayers to separate internal metadata from the geometry data

D

Convert all payloads to references before sharing

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Preparing an asset for external delivery typically involves **flattening** the layer stack into a single layer (removing composition complexity), rewriting internal asset paths to be relative, and packaging everything as a **USDZ archive**—a zip-based self-contained format designed for portability.

#11

COMPOSITION • 23% EXAM WEIGHT

Which statement best describes the difference between `Inherits` and `Specializes` in USD composition?

- A Inherits is used for mesh geometry; Specializes is used for shading graphs
- B Both arcs behave identically; the naming difference is purely semantic
- C Inherits allows a class prim's opinions to be overridden by local opinions on the inheriting prim, whereas Specializes keeps the base class's refinements as stronger
- D Specializes is used for payloads; Inherits is used for references

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Inherits** follows normal LIVERPS ordering—local opinions on the inheriting prim override the class prim. **Specializes** inverts this: the base class's specialized opinions propagate through to be stronger than the specializing prim's own local opinions. This makes Specializes useful for 'refinements that should always hold' like material look-dev overrides.

#12

COMPOSITION • 23% EXAM WEIGHT

A studio has 500 tree assets that all share the same wind animation cycle. Which approach uses the least memory and disk I/O at render time?

- A Copy the animation data into each tree asset's USD file
- B Use USD value clips to share a single animation layer across all tree instances
- C Bake animation into USDC crate format for each tree

D

Use variant sets with one variant per frame of animation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Value clips** allow many prims to share time-sampled animation data from a single layer file, reading only the relevant time range on demand. This is far more efficient than duplicating animation data per asset, and is the canonical USD approach for instanced procedural animation.

#13

COMPOSITION • 23% EXAM WEIGHT

A prim at `/World/Car` has a reference to `car.usda`, and inside that file the `Engine` prim is defined. A shot layer has an `over` at `/World/Car/Engine` to change its color. The artist reports the color isn't changing. What is the most likely cause?

A

The `over` specifier cannot target prims inside references

B

The prim path `/World/Car/Engine` is incorrect—it should be `/Car/Engine`

C

The `over` in the shot layer is authored in a layer that is weaker than the reference arc's internal layer

D

References do not allow any child prims to be overridden

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: If the shot layer's opinion on `/World/Car/Engine` is authored in a sublayer that is *weaker* than the layer stack the reference resides in, LIVERPS will not favor it. For local opinions to win over reference content, the override must be in a layer that is **stronger** (higher in the sublayer stack or the edit target).

#14

COMPOSITION • 23% EXAM WEIGHT

When is it NOT appropriate to use variant sets to structure an asset?

A

When the asset has multiple material finishes (e.g., polished vs. matte)

B When the asset has different levels of detail (LOD)

C When the differences between variants require entirely different prim hierarchies that are incompatible with each other

D When the asset needs both geometric and shading variations

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Variant sets work best when all variants share the same prim hierarchy—they swap opinions, not structure. If two 'variants' would require fundamentally **different hierarchies** (e.g., a car vs. a helicopter), references or separate assets are more appropriate. Forcing incompatible hierarchies into variants leads to fragile pipelines.

#15

COMPOSITION • 23% EXAM WEIGHT

A layer offset of `offset=24, scale=2` is applied to a reference. An attribute time sample originally at time 6 in the referenced layer will be evaluated at what time on the referencing stage?

A Time 36 on the stage ($6 \times 2 + 24$)

B Time 15 on the stage ($6 + 24 / 2$)

C Time 48 on the stage ($6 \times 2 \times 24 / 6$)

D Time 3 on the stage ($6 / 2$)

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A USD **layer offset** transforms layer-local time to stage time using the formula: $stageTime = localTime \times scale + offset$. So: $6 \times 2 + 24 = 36$. Layer offsets are the mechanism for placing the same animation at different points in time (e.g., staggering a shared walk cycle across multiple characters).

#16

COMPOSITION • 23% EXAM WEIGHT

Which of the following statements about USD encapsulation is correct?

- ☐ A Sublayers added inside a referenced layer are visible to the referencing stage and can have their opinions overridden
- ☒ B Composition arcs authored inside a referenced layer (nested sublayers, references within a reference) are encapsulated and cannot be overridden by the referencing stage
- ☐ C Encapsulation only applies to payloads, not to references
- ☐ D Encapsulation prevents prims inside a reference from being traversed by the referencing stage

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Encapsulation means that the internal composition of a referenced asset (its own sublayers, references, variant sets, etc.) is evaluated independently and the referencing stage cannot insert opinions into that internal composition. The referencing stage can override *values* on the composed prims, but cannot alter how the referenced asset is composed internally.

CONTENT AGGREGATION

#17

CONTENT AGGREGATION • 10% EXAM WEIGHT

A scene contains 10,000 tree prims all using ``instanceable = true`` and pointing to the same prototype. An artist wants to change the bark color on just one specific tree. What happens if they directly set ``primvars:displayColor`` on that single tree prim?

- ☐ A The change propagates to all trees sharing the prototype
- ☒ B The prim is automatically de-instanced and the change applies only to that tree, at the cost of losing instance optimization
- ☐ C USD ignores the override because instanced prims are read-only
- ☐ D The tree becomes invisible until the stage is reloaded

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When you author opinions directly on an instanced prim that cannot be represented in the prototype, USD will **de-instance** that prim—it no longer shares the prototype and its memory savings are lost. The change applies only to that individual prim. To avoid de-instancing, overrides should be authored via instance proxies or variant sets on the prototype.

#18

CONTENT AGGREGATION · 10% EXAM WEIGHT

What is the key functional difference between USD native (scenegraph) instancing and point instancing using ``UsdGeomPointInstancer``?

- ☒ A Native instancing works on entire subtrees of prims; point instancing efficiently places many copies of prototype geometry at positions defined by point arrays
- ☐ B Native instancing is only for geometry; point instancing is only for materials
- ☐ C Point instancing supports LIVERPS composition; native instancing does not
- ☐ D Native instancing requires USDC format; point instancing requires USDA format

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Native scenegraph instancing** marks entire prim subtrees as instanceable, sharing them via a prototype. **Point instancing** (``UsdGeomPointInstancer``) encodes many placements of geometry using compact position, orientation, and scale arrays — ideal for millions of copies (foliage, crowds) where per-instance scenegraph prims would be prohibitively expensive.

#19

CONTENT AGGREGATION · 10% EXAM WEIGHT

In a model hierarchy, which ``kind`` value should be used for a prim that acts as the root of a self-contained, referenceable asset (e.g., a single prop like a chair)?

- ☐ A group
- ☐ B assembly
- ☒ C component

D subcomponent

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **component** is the leaf-level model kind for a self-contained, referenceable asset. It may not contain other models as children. An **assembly** is a collection of components/groups. **Group** is for organizational containers of models. **Subcomponent** is for internal parts within a component that should not be referenced independently.

#20

CONTENT AGGREGATION • 10% EXAM WEIGHT



A set-dressing stage references the same building asset 50 times in a city block scene. What is the recommended way to ensure memory efficiency for this repeated asset?

A

Set `instanceable = true` on each referencing prim so USD can share a single prototype

B

Copy the asset's USD layer into a single merged layer with all 50 prims inlined

C

Use 50 separate payload arcs and unload them after set-dressing is complete

D

Convert the asset to USDA format so it is easier to duplicate

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Setting `**instanceable = true**` on prims that reference the same asset causes USD to create a single prototype and share it across all instances. This dramatically reduces memory usage for repeated assets. It is the primary mechanism for efficient aggregation of repeated content in large scenes.

#21

CONTENT AGGREGATION • 10% EXAM WEIGHT



To hide specific instances in a `UsdGeomPointInstancer` without removing them from the prototype list, which attribute should be used?

A

Set `visibility = invisible` on the PointInstancer prim itself

B Remove the points from the ``positions`` array for the instances you want hidden

C Use the ``invisibleIds`` attribute on the `PointInstancer` to list the instance indices to hide

D Set ``active = false`` on the `PointInstancer`'s prototype prim

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The ``invisibleIds`` attribute on ``UsdGeomPointInstancer`` allows specific instance indices to be hidden efficiently without restructuring the prototype list or positions array. This is the correct and performant way to selectively hide individual point instances.

#22

CONTENT AGGREGATION • 10% EXAM WEIGHT

An ``over`` prim is authored in an assembly stage to change the color of an instanced component. The component prim has ``instanceable = true``. What is the effect of this override?

A The override applies to all instances sharing that prototype

B The override is silently ignored because instanced prims cannot have local overrides

C USD automatically creates an instance proxy, and the override applies only to that specific instance, potentially de-instancing it

D The override forces the component to become a group kind

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Overrides on an **instance proxy** (the virtual prim that represents one instance) may cause de-instancing of that specific instance if the override cannot be represented on the shared prototype. The override applies to just that instance. Alternatively, if the override can be expressed via variants on the prototype, de-instancing can be avoided.

#23

CONTENT AGGREGATION • 10% EXAM WEIGHT

What is the primary benefit of using the reference/payload pattern (reference for the asset interface, payload for the heavy geometry) in an asset's internal structure?

- ☒ A It separates metadata and variant set definitions from heavy mesh data, allowing tools to query asset metadata without loading geometry
- ☐ B It ensures that material overrides are always applied after geometry is loaded
- ☐ C It prevents composition errors when the same asset is referenced multiple times
- ☐ D It converts geometry to USDC format automatically on first load

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **reference/payload pattern** places the asset's interface (metadata, variant sets, extentsHint) in the reference layer and heavy geometry in the payload. Tools and pipeline stages can open the stage, inspect asset metadata, and manage working sets—all *without* loading the heavy payload geometry. This is critical for performance in large-scale scenes.

CUSTOMIZING USD

#24

CUSTOMIZING USD • 6% EXAM WEIGHT

A pipeline needs to store proprietary simulation attributes on USD prims that are not part of any built-in schema. The attributes must be discoverable by other tools without parsing raw USD text. Which approach is most appropriate?

- ☐ A Prefix all attributes with `userProperties:` and document them in a wiki
- ☒ B Create a custom USD API schema that defines the attributes, and distribute the schema plugin to all tools
- ☐ C Store simulation data in USD metadata as string-encoded JSON
- ☐ D Write all simulation attributes as relationships pointing to a separate Scope prim

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **custom API schema** is the correct way to add proprietary attributes to USD prims in a way that is type-safe, discoverable, and distributable. The schema is compiled into a USD plugin and registered with the `PlugRegistry`, making it available to any tool in the pipeline that loads the plugin.

#25

CUSTOMIZING USD • 6% EXAM WEIGHT

What is the difference between an IsA schema and an API schema in USD?

- A** IsA schemas define the prim type (e.g., Mesh, Sphere, Xform); API schemas add additional properties to any prim without changing its type
- B IsA schemas are only for geometry; API schemas are only for materials
- C API schemas replace IsA schemas in modern USD; IsA schemas are deprecated
- D IsA schemas require C++ implementation; API schemas can only be implemented in Python

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **IsA schemas** define a prim's fundamental type—a prim IS a Mesh, IS a Sphere, IS an Xform. Only one IsA schema can apply to a prim. **API schemas** are applied additively ('prepend apiSchemas') and contribute additional properties without changing the prim's type. Multiple API schemas can be applied to the same prim.

#26

CUSTOMIZING USD • 6% EXAM WEIGHT

A studio wants USD to resolve asset paths like `asset:///props/chair.usda` using an internal asset database instead of the filesystem. Which USD extension point handles this?

- A A custom file format plugin that registers the `asset:///` URI scheme
- B** A custom `ArResolver` (Asset Resolver) plugin that translates logical paths to physical paths
- C A `UsdStage` sublayer that maps virtual paths to real file paths

D A variant set that stores alternate paths per environment

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **Asset Resolution (Ar)** library provides the ``ArResolver`` interface. A custom ``ArResolver`` plugin intercepts all asset path resolution calls in USD and can implement arbitrary logic—querying a database, applying version pinning, or translating logical URIs to filesystem paths. This is the correct extension point for custom asset management.

#27

CUSTOMIZING USD • 6% EXAM WEIGHT



A developer needs to create a new type of USD prim that represents a proprietary simulation domain object. The prim must appear as its own type when queried (e.g., ``prim.IsA()``). Which base class should the schema inherit from?

A UsdAPISchemaBase, because it allows the schema to be applied to any prim

B UsdSchemaBase, the root base class for all schemas

C UsdTyped, because it is the base class for all typed (IsA) schemas

D UsdGeomXformable, because all scene objects must be transformable

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **UsdTyped** is the base class for all *typed* (IsA) schemas—schemas that give a prim a specific type identity. API schemas, which are applied additively without changing prim type, inherit from ``UsdAPISchemaBase``. For a new prim type that can be tested with ``IsA<>``, the correct base is ``UsdTyped``.

#28

DATA EXCHANGE · 15% EXAM WEIGHT



A DCC exporter produces a USDA file with ``upAxis = "Y"`` and ``metersPerUnit = 0.01``. Another tool imports this file expecting ``upAxis = "Z"`` and ``metersPerUnit = 1.0``. What does USD do automatically during composition?

A

USD automatically reorients and rescales all geometry to match the importing tool's settings

B

USD does not automatically convert axis or units—the consuming application is responsible for interpreting and handling these metadata values

C

USD raises a fatal error and refuses to compose layers with mismatched axis settings

D

USD uses the root layer's `upAxis` and `metersPerUnit` to override all sublayers

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD treats ``upAxis`` and ``metersPerUnit`` as **advisory metadata**. It does not automatically transform geometry. Applications are responsible for reading these values and applying any necessary conversion. This is a common source of scale and orientation bugs when exchanging assets between DCCs.

#29

DATA EXCHANGE · 15% EXAM WEIGHT



A technical director must decide between storing a large scene as USDA (ASCII) or USDC (binary crate). Which statement correctly describes the trade-offs?

A

USDA is faster to read and write; USDC is human-readable for debugging

B

USDC offers better read performance and smaller file sizes; USDA is human-readable and easier to diff/version-control

C

Both formats have identical performance; the choice is purely aesthetic

D

USDC supports time samples; USDA does not

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USDC (crate format) is a binary, memory-mapped format designed for performance—faster reads, smaller files, and efficient random access to time samples. **USDA** is human-readable ASCII, making it invaluable for debugging, version control diffs, and hand-authoring small files. Large production assets should use USDC; interface/config layers can use USDA.

#30

DATA EXCHANGE • 15% EXAM WEIGHT

A studio is building a pipeline validator for assets exported from a DCC. Which USD tool and API is specifically designed for compliance checking of USD assets?

A

`usdcat` — converts between layer formats and validates syntax

B

`usdzip` — packages assets and validates embedded paths

C

`usdchecker` — runs a suite of compliance checks on a USD layer or USDZ file

D

`usdview` — visualizes the composed stage and reports composition errors

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `usdchecker` is the USD toolset's dedicated compliance validator. It runs a set of rules (checking for valid defaultPrim, correct extent attributes, proper material bindings, etc.) and reports errors and warnings. It is the starting point for building automated asset validation in a DCC export pipeline.

#31

DATA EXCHANGE • 15% EXAM WEIGHT

A studio is implementing a round-trip export/import of a character between Maya and USD. After export to USD and reimport to Maya, the character's normals are flipped. What is the most likely cause?

A

USDA format does not support normal vectors

B

The `upAxis` or winding order convention differs between Maya and the USD exporter, causing an axis flip that inverts face normals

C The ``metersPerUnit`` value changed, scaling the normals incorrectly

D USD stores normals as relationships, not attributes, causing Maya's importer to misinterpret them

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Normal flipping on round-trip is almost always caused by a mismatch in **coordinate handedness or winding order**—often combined with an ``upAxis`` flip (Y-up vs Z-up). When geometry is reoriented to match a different axis convention, face winding is inverted, which flips surface normals. The exporter must correctly handle axis conversion.

#32

DATA EXCHANGE • 15% EXAM WEIGHT

What is the conceptual difference between a USD exporter and a USD converter?

A An exporter saves DCC-native data to USD format; a converter transforms data between USD and another 3D format (e.g., glTF)

B An exporter uses Python; a converter uses C++

C An exporter only handles geometry; a converter handles all data types including materials and animation

D There is no difference—the terms are interchangeable in OpenUSD documentation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: An **exporter** translates data from a DCC application's internal representation into USD. A **converter** transforms data between two file formats (e.g., USD to glTF, USD to FBX) outside any specific DCC application. Both involve *conceptual data mapping*—understanding how concepts in one system map to USD (or vice versa).

#33

DATA EXCHANGE • 15% EXAM WEIGHT

Which USDA attribute would you check to determine whether a given ``usd`` file is actually stored in ASCII or binary crate format?

A

Read the ``format`` metadata from the stage's root layer

B

Check the file extension: ``.usda`` = ASCII, ``.usdc`` = binary, ``.usd`` = either—inspect the magic bytes or use ``usdcat --flatten``

C

Binary and ASCII cannot be distinguished without fully parsing the file

D

Check whether ``crate = true`` appears in the file header

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The ``.usd`` extension is ambiguous—it can be either ASCII or binary. The reliable way to determine format is to inspect the **magic bytes** at the start of the file (``PXR-USDC`` for binary crate, ``#usda`` for ASCII), or to use the ``usdcat`` command which handles both and can convert between them.

#34

DATA EXCHANGE • 15% EXAM WEIGHT



A USDZ file is described as a 'self-contained USD package.' What makes USDZ distinct from a regular USD layer?

A

USDZ is a compressed zip archive that bundles USD layers and their referenced assets (textures, etc.) into a single file

B

USDZ is a binary format that is more compact than USDC

C

USDZ is read-only and cannot be modified after creation

D

USDZ uses a different schema system than standard USD

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **USDZ** is a zip-based archive that packages one or more USD layers along with all referenced assets (textures, materials, etc.) into a single portable file. The zip is *uncompressed* for memory-mappable access. This makes USDZ ideal for sharing self-contained assets for AR (e.g., Apple Reality Composer) or external delivery.

#35

DATA EXCHANGE · 15% EXAM WEIGHT



When writing a USD importer for a DCC that receives geometry from an external source, what is the recommended approach for handling geometry that uses a different coordinate up-axis than USD expects?

A

Rotate all incoming point positions by the necessary matrix before writing them to USD

B

Write the ``upAxis`` metadata in the root layer to reflect the source coordinate system, and let all consuming tools handle the difference

C

Skip all geometry that does not match the expected up-axis

D

Store the transformation in a variant set so users can switch between axis conventions

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The recommended practice is to write the correct ``upAxis`` metadata in the USD layer's header to accurately document the source convention, rather than baking a rotation into the geometry data. This preserves the original data fidelity and lets downstream consumers apply the necessary transform. Baking in a compensating rotation can cause compounding errors.

#36

DATA EXCHANGE · 15% EXAM WEIGHT



A data mapping document needs to describe how a proprietary material model maps to USD. The material has an 'anisotropy direction' attribute that has no direct equivalent in ``UsdPreviewSurface``. What is the best strategy?

A

Ignore the attribute entirely since USD cannot represent it

B

Store it as a custom attribute using the ``userProperties:`` namespace and document the mapping

C

Approximate it using the `UsdPreviewSurface`roughness`` attribute with a comment

D

Store the value in stage-level metadata as a string

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: For attributes that have no USD equivalent, the best practice is to store them as **custom attributes** in the `userProperties:` namespace (or a pipeline-specific namespace), and **document the conceptual mapping** so importers and exporters can correctly interpret them. This preserves round-trip fidelity without breaking schema compliance.

#37

DATA EXCHANGE • 15% EXAM WEIGHT

When converting a USD scene to glTF, which USD concept does NOT have a direct glTF equivalent and requires special handling or loss of fidelity?

A

Mesh vertex positions (points array)

B

PBR material parameters (roughness, metallic)

C

USD composition arcs (references, variants, payloads)

D

UV texture coordinates stored as primvars

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: glTF is a **flattened delivery format**—it has no concept of composition arcs, variant sets, payloads, or sublayers. When converting from USD to glTF, the stage must be flattened (composition evaluated) and the resulting geometry/material data exported. Composition arcs are inherently lost in the conversion—this is a significant fidelity trade-off.

#38

DATA EXCHANGE • 15% EXAM WEIGHT

A USD validator finds that an exported mesh's `extent` attribute does not match the actual bounding box of its `points`. Why does this matter for the pipeline?

A

The `extent` attribute only affects rendering quality, not pipeline functionality

B

An incorrect `extent` causes USD's frustum culling and bounding box queries to return wrong results, breaking LOD selection, camera framing, and scene optimization tools

C

The `extent` attribute is purely cosmetic metadata and has no functional effect

D

USD automatically recalculates `extent` at runtime so mismatches are corrected silently

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: The **`extent`** attribute is used by renderers, layout tools, and pipeline utilities for frustum culling, bounding box queries, and automatic camera framing. An incorrect extent means these systems will make wrong decisions—objects may be culled when visible, or bounding boxes in UI will be wrong. Exporters must always recompute and write accurate extents.

#39

DATA MODELING • 15% EXAM WEIGHT



A primvar is authored on a mesh with interpolation **`faceVarying`**. The mesh has 100 faces, each a quad (4 vertices). How many values must the primvar array contain?

A

100 — one per face

B

400 — one per face-vertex (4 vertices × 100 faces)

C

The number of unique vertices in the mesh

D

1 — primvars with **faceVarying** interpolation store a single averaged value

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: **faceVarying** interpolation stores one value per face-vertex—that is, per each corner of each face (even if vertices are shared). For 100 quads: 4 vertices × 100 faces = **400** values. This is the finest-grained interpolation and is used for attributes like UVs that can differ at shared vertices.

#40

DATA MODELING • 15% EXAM WEIGHT



Which value type should be used to store a 3×3 rotation matrix as a USD attribute?

A

float3[] — an array of three float3 vectors

B matrix3d — a double-precision 3×3 matrix type provided by Sdf

C token — encoded as a string representation

D quatd — a double-precision quaternion

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's **Sdf type system** includes `matrix3d` (3×3 double-precision matrix) and `matrix4d` (4×4 double-precision matrix) as native value types. Using the correct matrix type ensures proper interpolation, schema validation, and cross-application interoperability. A `quatd` is an alternative for rotations but represents orientation differently.

#41

DATA MODELING · 15% EXAM WEIGHT



What is the purpose of `custom` keyword when declaring an attribute in USDA?

A It marks the attribute as mandatory for the schema to be valid

B It indicates that the attribute is not defined in any applied schema—it is an ad-hoc attribute added outside schema definitions

C It enables the attribute to accept arbitrary value types at runtime

D It prevents the attribute from being composed across layers

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The `custom` keyword in USDA marks an attribute as user-defined—it does not belong to any applied typed or API schema. While this provides flexibility, it is generally discouraged in production pipelines because custom attributes are not type-validated by schemas. Prefer defining a proper API schema for structured pipeline data.

#42

DATA MODELING · 15% EXAM WEIGHT



An attribute has a default value of `1.0` and a time sample at time 10 of `5.0`. What value does USD return when querying this attribute at time 5 (assuming

linear interpolation is enabled)?

- A** 1.0 — the default value is always used when no time sample exactly matches
- B 5.0 — the nearest time sample value is used
- C 3.0 — USD interpolates between the default value (1.0) and the time sample (5.0) at the midpoint
- D The default value is not used for time-sampled attributes; USD returns `None`

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In USD, **default values do not participate in time sample interpolation**. They are the 'held' value for time codes before the first time sample, after the last time sample, or when there are no time samples at all—but USD does NOT interpolate between a default value and a time sample. At time 5 (before the only time sample at 10), USD returns the **default value 1.0**.

#43

DATA MODELING · 15% EXAM WEIGHT

A developer is choosing between representing mesh UVs as a `texCoord2f[]` attribute versus a `texCoord2f[]` primvar. Why are primvars preferred for UV coordinates?

- A Primvars are faster to read than regular attributes at render time
- B** Primvars carry interpolation metadata, support indexed arrays for compression, and are automatically inherited by child prims through the primvar inheritance mechanism
- C Regular attributes cannot store 2D float arrays in USD
- D Primvars are the only attribute type that can be time-sampled

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Primvars (primitive variables) have several advantages over regular attributes for per-geometry data: they carry *interpolation metadata* (vertex, faceVarying, etc.) so renderers know how to sample them, they support *indexed arrays* to avoid value duplication, and they

inherit down the prim hierarchy. These properties make them essential for UV maps, display colors, and other renderer-consumed per-geometry data.

#44

DATA MODELING · 15% EXAM WEIGHT

Custom metadata in USD is distinct from regular attributes. Which of the following is a correct statement about custom metadata?

A

Custom metadata is stored on prims and can be time-sampled like attributes

B

Custom metadata must be declared in a schema plugin's `plugInfo.json` before it can be authored in a layer

C

Custom metadata can be authored freely on any prim without prior registration, but it is limited to string values

D

Custom metadata appears in the composed value stack and follows LIVERPS strength ordering like attributes

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: While *some* metadata can be authored freely, **custom metadata** that needs to be validated and properly typed must be declared in a schema plugin's `plugInfo.json`. Metadata is not time-sampable, does not appear in the property stack like attributes, and follows different composition rules. Unregistered metadata may appear as 'unregistered' and behave unexpectedly across tools.

#45

DATA MODELING · 15% EXAM WEIGHT

A mesh is authored with points at one position, but after updating the 'points' attribute with new positions, the mesh appears incorrectly culled in the renderer. What attribute was likely not updated?

A

faceVertexIndices

B

normals

C

extent

D primvars:displayColor

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The ``extent`` attribute stores the axis-aligned bounding box of the mesh. Renderers use it for frustum culling. When ``points`` are updated to new positions, ``extent`` must be recalculated and updated accordingly. If ``extent`` still reflects the old positions, the renderer may incorrectly cull the mesh even when it is within the camera's view frustum.

#46

DATA MODELING · 15% EXAM WEIGHT

Which primvar interpolation mode stores exactly one value for the entire prim, regardless of how many vertices or faces it has?

A vertex

B faceVarying

C uniform

D constant

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Constant interpolation stores a single value that applies to the entire primitive—one value for the whole mesh. ``vertex`` stores one value per vertex, ``faceVarying`` stores one per face-vertex, and ``uniform`` stores one per face. ``constant`` is used for prim-wide values like a single display color for the entire object.

#47

DATA MODELING · 15% EXAM WEIGHT

A relationship in USD differs from an attribute in a fundamental way. Which statement correctly describes this difference?

A Relationships store typed values (float, int, token); attributes store prim paths

B Relationships point to other prims or properties by path; attributes store typed values

C Relationships are time-sampled; attributes are not

D Attributes can compose across layers; relationships cannot

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Attributes** store typed values (float, int, string, color, matrix, etc.) on a prim. **Relationships** store targets—paths to other prims or properties in the scenegraph. The canonical use of a relationship is `material:binding`, which points from a mesh prim to a material prim. Relationships cannot hold data values.

DEBUGGING

#48

DEBUGGING • 11% EXAM WEIGHT

An artist reports that a material override authored in `shot.usda` is not visible on the final render, even though the file is correctly included as a sublayer. After investigating, you discover `shot.usda` is the bottom-most sublayer. What is the likely issue?

A Sublayers at the bottom of the stack have the weakest opinions; the override is being trumped by a stronger sublayer

B Material overrides cannot be authored in sublayers—they must be in the root layer

C The USDA format does not support material overrides in sublayers

D The override requires `prepend references` to take effect in a sublayer

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In a sublayer stack, **layers listed earlier (closer to the top/root) have stronger opinions**. A sublayer at the bottom has the weakest opinion strength. If a stronger sublayer (or the root layer itself) has a conflicting opinion on the same attribute, the bottom sublayer's value will be ignored. The fix is to move `shot.usda` higher in the sublayer order.

#49

The `usdview` LayerStack tab is useful for debugging which type of issue? ▲

A

Viewing all shaders compiled for a specific material

B

Seeing which layers contribute opinions to a selected prim and in what order, to trace where a value is coming from

C

Profiling USD load times per layer

D

Viewing all animation time samples on a selected attribute

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **LayerStack tab** in `usdview` displays the full layer stack contributing to a selected prim, showing each layer's opinions and their order. This is the essential debugging tool for tracing which layer is responsible for a specific value, why an expected override isn't taking effect, or why an unwanted opinion is winning.

#50

DEBUGGING • 11% EXAM WEIGHT ▲

A stage opens successfully but a referenced mesh appears invisible. Checking the composed stage, the mesh prim exists but `visibility = invisible`. The override was not intentionally set. Where should you look first?

A

Check if the mesh's `active` metadata was set to false in any layer

B

Use `UsdProperty::GetPropertyStack()` to find all layers contributing a `visibility` opinion, then identify which layer is setting it to invisible

C

Delete the mesh prim from the stage and re-reference the asset

D

Check if the mesh's `kind` is set to `subcomponent`, which hides prims by default

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `UsdProperty::GetPropertyStack()` returns the ordered list of property specs contributing to an attribute, from strongest to weakest, across all layers. This is the correct API

approach to trace an unwanted `visibility` value—it identifies exactly which layer file is authoring the invisible opinion so it can be fixed or muted.

#51

DEBUGGING · 11% EXAM WEIGHT

What does `SdfChangeBlock` do, and when should it be used?

- A It prevents any changes from being authored on a layer after it is opened, making it effectively read-only
- B It batches multiple scene description changes together to minimize the number of change notifications sent to listeners, improving performance for bulk edits**
- C It locks a specific prim so it cannot be overridden by downstream layers
- D It flushes pending layer changes to disk to prevent data loss

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `SdfChangeBlock` defers USD's change processing notifications until the block goes out of scope. When making many changes to scene description (e.g., setting 10,000 attribute values), wrapping the operations in an `SdfChangeBlock` prevents the overhead of sending and processing individual change notifications for each edit—significantly improving bulk-edit performance.

#52

DEBUGGING · 11% EXAM WEIGHT

A USD pipeline tool is reporting unexpected asset resolution failures for paths that worked previously. The tool uses a custom `ArResolver`. What is the most effective first debugging step?

- A Re-export all assets from the DCC in USDC format
- B Enable `TfDebug` categories related to asset resolution (e.g., `AR\_RESOLVER\_INIT`) to trace the resolver's decision logic**
- C Switch from USDA to USDC format for all affected layers

D

Flatten the stage to remove all composition before testing resolution

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `TfDebug` is USD's built-in debugging facility. Enabling relevant debug categories (set via `TF_DEBUG` environment variable or `TfDebug::SetDebugSymbolsByName()`) causes USD to print detailed trace information about the system being debugged. For asset resolution issues, categories like `AR_RESOLVER_INIT` expose the resolver's initialization and path-lookup behavior.

#53

DEBUGGING · 11% EXAM WEIGHT



A character prim composed from multiple layers shows a translate value of `(0, 0, 0)` on the stage, but the artist says they set it to `(10, 0, 0)` in their shot layer. The shot layer is confirmed to be loaded. Which LIVERPS-related reason is most likely?

A

The `(0, 0, 0)` value must be authored by a local opinion in a stronger sublayer that overrides the artist's shot layer

B

USD clamps translate values to `(0, 0, 0)` for prims that are inside a reference

C

The shot layer's translate must be in a payload for it to take effect

D

The artist authored the translate on the wrong prim type—Xform prims cannot have translates

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The most common cause of an override not taking effect is a **stronger layer opinion** overriding it. The `(0, 0, 0)` value is likely authored in a sublayer that is higher (stronger) in the stack than the artist's shot layer. Checking the LayerStack tab in usdview or using `GetPropertyStack()` would reveal which layer is winning.

#54

DEBUGGING · 11% EXAM WEIGHT



When a USD stage takes a very long time to open, which approach is most appropriate for identifying the bottleneck?

A Convert all layers to USDA format to make them human-readable for manual inspection

B Use `TfMallocTag` or the USD `Trace` profiling system to measure time spent in specific subsystems (e.g., asset resolution, composition, payload loading)

C Remove all variant sets from the scene description to simplify composition

D Open the stage with a different Python version to check for interpreter overhead

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD includes a **Trace profiling system** and `TfMallocTag` for memory profiling. The Trace system records hierarchical timing data for USD operations, allowing developers to pinpoint exactly which subsystem (asset resolution, composition evaluation, payload loading, etc.) is consuming the most time during stage opening.

#55

DEBUGGING · 11% EXAM WEIGHT

A pipeline reports that after muting a specific layer using `UsdStage::MuteLayer()`, some prims disappear entirely from the stage. What is the most likely explanation?

A Muting a layer corrupts the stage's composition cache

B The muted layer was the only layer defining those prims (e.g., with a `def` specifier), so removing it causes those prims to no longer exist in the composed stage

C Muting a layer removes all of its sublayers recursively from composition

D USD does not support muting layers that contain `def` specifiers

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When a layer is muted, all of its scene description is excluded from composition. If a prim was **only defined** (`def`) in the muted layer—with no other layer contributing a `def` or `over`—the prim ceases to exist in the composed stage. Muting is a non-destructive debugging tool, but its effects on composition can be significant if the muted layer is load-bearing.

#56

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline architect wants to design a USD layer strategy so that a lighting artist can override just the light intensities for a shot, without affecting the asset definitions or set dressing. Which layer strategy achieves this?

- A Package all lighting changes inside a variant set in the asset files
- B Add a dedicated shot lighting sublayer above the set-dressing layer in the root stage's sublayer stack, where lighting opinions can override the underlying layers**
- C Use a payload arc for each light so its properties can be unloaded independently
- D Flatten the entire stage before the lighting artist works on it

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The standard pipeline pattern is to have **a dedicated shot layer** for each department (layout, animation, lighting, FX), added as a sublayer with appropriate strength ordering. A lighting sublayer positioned above set-dressing layers gives lighting opinions priority, allowing non-destructive light intensity overrides without touching asset or set-dressing definitions.

#57

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline must support both ``metersPerUnit = 1.0`` (meters) and ``metersPerUnit = 0.01`` (centimeters) for assets coming from different DCCs. Which practice ensures correct behavior in the composed stage?

- A Always set ``metersPerUnit = 1.0`` in every layer and bake a scale into the geometry
- B Document the ``metersPerUnit`` value in each asset's layer header accurately, and apply a compensating scale xform when referencing assets with a different unit into the stage**
- C Use variant sets to store geometry at both scales and let the consumer choose
- D USD ignores ``metersPerUnit`` so no special handling is needed

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: The recommended practice is to **accurately record `metersPerUnit`** in each asset and apply an appropriate **compensating scale transform** (via an Xform prim) when referencing assets with different units into a unified stage. This preserves original geometry data fidelity while ensuring correct spatial scale relationships in the assembled scene.

#58

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline tool needs to iterate over every prim in a stage that has a specific API schema applied. Which is the correct USD API pattern?

A

Traverse the stage and check `prim.GetTypeName()` for the schema name

B

Use `UsdStage::Traverse()` and filter prims using `prim.HasAPI()`

C

Query the root layer's `subLayers` list and find layers that define the schema

D

Use `UsdPrimRange` with a `UsdTraverseInstanceProxies` predicate only

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: `UsdStage::Traverse()` (or `UsdPrimRange`) combined with `prim.HasAPI()` is the correct pattern. `HasAPI` checks whether a specific API schema has been applied to a prim. `GetTypeName()` only returns the `IsA` schema type, not applied API schemas. For typed schemas, `prim.IsA()` is the correct check.

#59

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A studio is adopting USD and needs to document their asset structure guidelines. Which of the following is a key principle recommended by the ASWF Asset Structure Guidelines?

A

All assets should be stored as a single monolithic USD file to minimize file count

B

Asset interfaces should expose variant sets and metadata at the top of the file structure, with heavy geometry deferred via payloads

C

Assets should never use variant sets, as they complicate pipeline tooling

D

All pipeline stages should write opinions to the same single layer file

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **ASWF Asset Structure Guidelines** (and the Principles of Scalable Asset Structure in OpenUSD) emphasize: expose asset interface (metadata, variants) at the top layer, defer heavy geometry via **payloads**, modularize workstreams into separate layers, and maintain a consistent, predictable hierarchy. This enables pipeline tools to query assets efficiently without loading all geometry.

#60

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT ▲

A pipeline uses ``UsdNotice`` to respond to stage changes. An artist rapidly edits 200 attributes in quick succession, and the change-notification callback is called 200 times, slowing the UI. What is the best solution?

A

Disable ``UsdNotice`` entirely during batch edits

B

Wrap the 200 attribute edits in an ``SdfChangeBlock`` so that only one batch of change notifications is sent when the block exits

C

Throttle the callback to fire at most once per second

D

Convert the stage to a new format before making bulk edits

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Wrapping bulk edits in an ``SdfChangeBlock`` defers all change processing until the block exits, at which point a single consolidated change notice is sent. This is the correct and idiomatic USD solution for preventing excessive notification overhead during programmatic bulk edits, such as setting many attributes in a loop.

#61

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT ▲

A pipeline TD wants to flatten a stage to remove all composition arcs and produce a single-layer USD file for delivery. Which API call achieves this?

A

``UsdStage::Save()``

B

``UsdStage::Flatten()``

C

``UsdStage::Export()`` with the ``addSourceFileComment=False`` option

D

``UsdStage::GetRootLayer().Flatten()``

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdStage::Flatten()`` evaluates all composition on the stage and returns a new ``SdfLayer`` containing all the resolved scene description in a single, flat layer—all composition arcs are baked in and their internal structure is removed. This is the correct API for producing a composition-free delivery asset.

#62

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT ▲

Which of the following describes a valid use case for the ``Flatten`` operation in a production pipeline?

A

Flattening during daily artist iteration to simplify the layer stack for faster development

B

Flattening a complex multi-layer asset for external delivery to a vendor who does not have access to the pipeline's internal asset resolver

C

Flattening after each sublayer is added to improve composition performance

D

Flattening is required before any USDC binary encoding can occur

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Flattening is most appropriate for **external delivery** when the recipient lacks access to the pipeline's internal tools (e.g., a custom ``ArResolver`` or proprietary schema plugins).

Flattening bakes all composition and resolves all internal dependencies into a self-contained layer. It should NOT be used during active development as it destroys the non-destructive layering structure.

#63

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline validates asset paths before publishing. Which property of USD paths makes this validation non-trivial?

A

USD paths always use absolute filesystem paths, so validation is straightforward

B

USD supports multiple asset path formats including relative paths, search paths, and custom URI schemes resolved by ArResolver—validation must account for all of these

C

USD only allows alphanumeric characters in paths, simplifying validation

D

Asset paths in USD are always validated automatically by the SdfLayer during authoring

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD asset paths can be **relative filesystem paths, absolute paths, or arbitrary URI strings** that a custom 'ArResolver' translates. This means path validation cannot simply check for file existence—it must understand the resolver in play, handle search paths, and validate against whatever resolution scheme the pipeline uses. This complexity is why asset path validation is a first-class pipeline concern.

#64

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A studio's animation pipeline generates USD layers with velocity attributes ('velocities') on meshes in addition to position time samples. What is the primary reason for including velocity data?

A

Velocities are required for USD stages to compute correct bounding boxes

B

Velocities allow motion blur to be computed at render time using fewer position time samples, reducing file size and improving I/O performance

C USD uses velocity attributes to interpolate between position time samples during composition

D Velocities are needed for USD's physics simulation system to function

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In USD, **velocity attributes** on geometry allow renderers to compute *motion blur* with fewer stored time samples. Instead of needing positions at many sub-frame time steps, a renderer can extrapolate motion-blurred positions from a single frame's position plus its velocity, dramatically reducing the number of time samples that need to be stored and read.

#65

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A newly hired pipeline engineer must choose between using sublayers and references to combine layers from two different departments into a single stage. Which factor most clearly distinguishes when to use sublayers versus references?

A Sublayers combine layers at the same prim namespace level; references graft an asset's prim hierarchy under a specific prim in the referencing stage

B Sublayers support time-sampled data; references do not

C References allow variant sets; sublayers do not allow variant sets

D Sublayers are for materials only; references are for geometry

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Sublayers** merge content at the same namespace level—prims from sublayers appear as if they were all authored in the root layer. **References** graft another layer's prim hierarchy *under a specific prim* in the referencing stage. Use sublayers for 'overlays' that span the whole stage (e.g., a lighting pass); use references for including a specific asset at a specific point in the scene hierarchy.

#66

VISUALIZATION · 8% EXAM WEIGHT



A `UsdPreviewSurface` material needs to read diffuse color from a UV-mapped texture. What is the correct node connection pattern?

A

Connect a `UsdUVTexture` node's `rgb` output directly to the `UsdPreviewSurface`'s `diffuseColor` input; connect a `UsdPrimvarReader_float2` to the `UsdUVTexture`'s `st` input to supply UV coordinates

B

Connect the texture file path string directly to the `UsdPreviewSurface`'s `diffuseColor` input

C

Set the `primvars:displayColor` attribute on the mesh and reference it via a `UsdPreviewSurface` input

D

Use a `UsdShadeNodeGraph` to store the texture path and bind it to the material's surface output

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The standard `UsdPreviewSurface` shading network connects: a `UsdPrimvarReader_float2` (reads UV primvar) → `UsdUVTexture` (samples texture at those UVs) → `UsdPreviewSurface` `diffuseColor` input. This chain is the canonical way to drive material color from a UV-mapped texture in the `UsdPreviewSurface` shading model.

#67

VISUALIZATION · 8% EXAM WEIGHT



What is the purpose of `UsdGeomSubset` in the context of material binding?

A

It divides a mesh into separate files for more efficient loading

B

It defines a subset of a mesh's faces (or other elements) so that different materials can be bound to different parts of the same mesh prim

C

It creates LOD levels for a mesh that are automatically selected by the renderer

D

It is used to group multiple mesh prims together under a single material

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdGeomSubset`` partitions a mesh's faces into named subsets, each represented as a child prim containing a list of face indices. Materials can then be bound to individual subsets using ``UsdShadeMaterialBindingAPI``, allowing a single mesh prim to display different materials on different faces—essential for assets with multiple material regions.

#68

VISUALIZATION · 8% EXAM WEIGHT

A mesh prim in USD does not have ``MaterialBindingAPI`` applied, but a parent Xform prim does have a material binding. What material is used for the mesh at render time?

- A No material is applied—bindings do not inherit and the mesh renders with a default grey shader
- B The material bound on the closest ancestor prim (the parent Xform) is used through the material binding inheritance mechanism**
- C The mesh searches all sibling prims for a valid material binding
- D USD picks the first material defined anywhere in the stage

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Material bindings in USD **inherit down the prim hierarchy**. If a prim does not have its own material binding, USD walks up the ancestor chain and uses the binding from the nearest ancestor that has one. This allows a single binding on an Xform to apply to all geometry descendants, simplifying material assignment for complex hierarchies.

#69

VISUALIZATION · 8% EXAM WEIGHT

A ``UsdLuxDomeLight`` is added to a stage to provide environment lighting. The artist wants to orient the environment map so a specific feature (e.g., the sun) points in a particular direction. How is this achieved?

- A Modify the ``textureFormat`` attribute on the DomeLight to specify a rotation offset
- B Apply a rotation transform to the DomeLight prim using standard Xform operations (xformOp:rotateXYZ)**

C

Set the ``guideRadius`` attribute to control light direction

D

DomeLights cannot be rotated—the texture must be pre-rotated before use

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A ``UsdLuxDomeLight`` inherits from ``UsdGeomXformable``, meaning it can have **standard transform operations** applied to it. Rotating the DomeLight prim using ``xformOp:rotateXYZ`` (or equivalent) rotates the environment map sphere accordingly, allowing precise control over the orientation of features in the HDRI/environment texture.

#70

VISUALIZATION • 8% EXAM WEIGHT



A mesh renders with a uniformly flat appearance—no shading variation despite having a complex shape. The material uses a physically-based shader. What is the most likely missing data?

A

The mesh is missing a ``primvars:displayColor`` primvar

B

The mesh is missing surface normals—without them, renderers cannot compute correct lighting angles and produce flat shading

C

The mesh is missing a ``kind`` metadata value

D

The `UsdPreviewSurface` roughness attribute is not set

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A physically-based renderer computes shading by evaluating the angle between the surface **normal** and the light direction. If a mesh has no authored normals, the renderer may compute flat face normals (producing faceted shading) or fall back to a zero vector, resulting in completely flat, un-shaded appearance. Missing or corrupt normals are the most common cause of unexpected flat rendering on complex meshes.